

Custom Topology Progress

Phase 0: Baseline Coverage

(Baseline: An already existing, well-known implementation that you compare against.)

Injection Rate

The **injection rate** is the average rate at which each node injects packets into the network, expressed as **packets/node/cycle** (or flits/node/cycle, depending on the simulator). It represents the offered network load.

- **Low injection rate:** Light traffic, minimal congestion.
- **High injection rate:** Heavy traffic, increased contention and congestion.
- As the injection rate approaches the network's capacity, latency rises sharply due to saturation.

Latency (Cycles)

Latency is the average number of **clock cycles** required for a packet to travel from its source node to its destination node.

It includes:

- Router processing delay,
- Link traversal delay,
- Buffer waiting time,
- Contention due to network congestion.

Lower latency indicates faster packet delivery and better network performance.

Average Hop Count (Hops Avg)

The **average hop count** is the mean number of routers (or links) traversed by packets from source to destination.

- **Lower hop count** generally reduces communication delay and dynamic network energy.
- Hop count depends primarily on the **network topology** and **routing algorithm**, rather than the traffic load.

In Booksim, **Hops Avg** is commonly used as a **NoC energy proxy**, since communication energy is approximately proportional to the number of links and routers traversed.

Task1: Verify shipped baselines run on matrix traffic: mesh4x4, torus4x4, fly4



Topology: fly4

injection_rate	latency_cycles	hops_avg
0.002	16.2186	2
0.005	16.4744	2
0.01	17.0945	2
0.02	20.004	2
0.03	292.534	2

Topology: mesh4x4

0.002	20.0326	2.7814
0.005	20.7332	2.88619
0.01	21.8697	2.92345
0.02	23.1712	2.83997
0.03	163.962	2.95006

Topology: torus4x4

0.002	18.3581	2.21395
0.005	18.9963	2.2556
0.01	19.4202	2.24919
0.02	21.1919	2.26194
0.03	183.03	2.2727

All baseline topologies executed successfully using the Timeloop-generated traffic matrix. Fly achieved the lowest latency under light load, Torus reduced average hop count compared to Mesh, and all topologies experienced a sharp latency increase near an injection rate of 0.03, indicating network saturation.

Task2: Add fattree16.cfg (k=4, n=2, routing nca)



injection_rate	latency_cycles	hops_avg
0.002	14.8279	1.74419
0.005	15.1325	1.7265
0.01	16.0277	1.77524
0.02	17.8217	1.76433
0.03	175.011	1.76224

FatTree achieved the lowest average hop count among the evaluated baselines (~1.75), which resulted in the lowest communication energy proxy and the lowest latency before saturation.

Task3: Add flattened butterfly flatfly16.cfg (ran_min routing; ugal needs extra VCs)



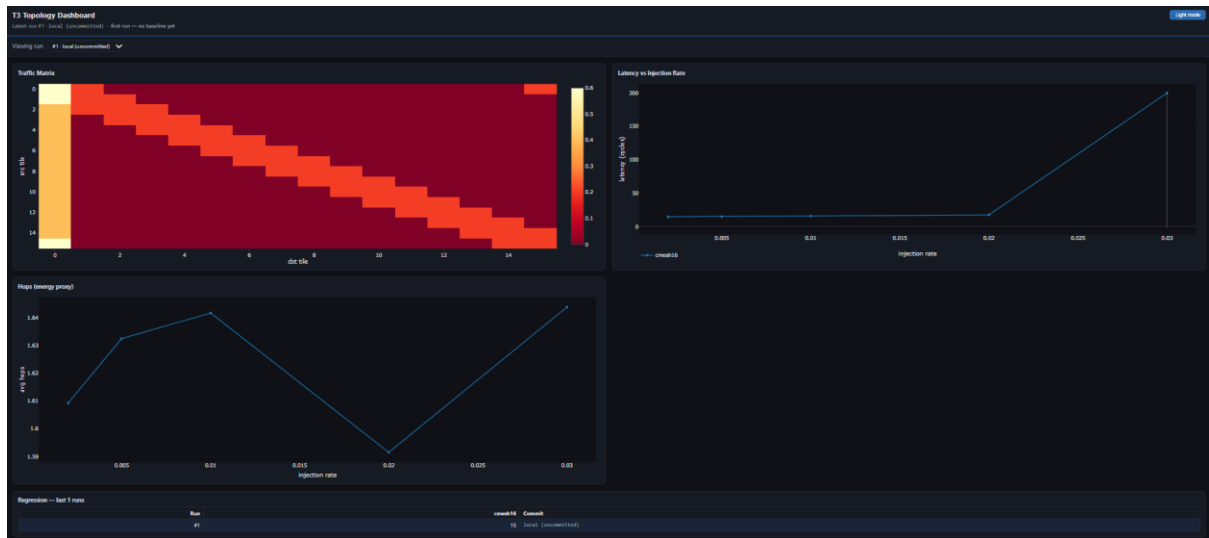
topology	injection_rate	latency_cycles	hops_avg
flatfly16	0.002	16.9209	2.02326
flatfly16	0.005	17.6068	2.06838
flatfly16	0.01	18.2166	2.08632
flatfly16	0.02	20.3439	2.07245
flatfly16	0.03	114.577	2.09349
flatfly_knc	0.002	19.0419	2.02326
flatfly_knc	0.005	19.6197	2.06838
flatfly_knc	0.01	20.5961	2.08632
flatfly_knc	0.02	22.6537	2.07245
flatfly_knc	0.03	125.54	2.08222
flatfly_ugal	0.002	16.9209	2.02326
flatfly_ugal	0.005	17.7073	2.0812
flatfly_ugal	0.01	18.2769	2.10098
flatfly_ugal	0.02	20.3846	2.1051
flatfly_ugal	0.03	134.074	2.38314

The FlatFly topology provides inherently low latency due to its high radix and low diameter. With `ran_min` routing, packets always follow shortest paths, resulting in minimum hop count and lower latency under light traffic. However, minimal routing causes congestion concentration and earlier saturation. UGAL routing improves scalability by dynamically selecting between minimal and non-minimal paths based on congestion information. Although UGAL increases hop count and routing complexity, it achieves higher saturation throughput and better load balancing under heavy traffic.

Reducing FlatFly parameters from $x=4, y=4$ to $x=1, y=1$ decreases router radix, network dimensions, and available routing paths. Although the smaller topology may exhibit lower absolute latency due to reduced network size, it suffers from reduced bandwidth, lower path diversity, and earlier saturation. Under `ran_min` routing, the advantage of random minimal path selection is reduced because fewer alternative minimal paths are available, leading to higher congestion.

flatfly16 has `ran_min` routing, $x=4, y=4$; flatfly_knc has `ran_min` routing, $x=1, y=1$; flatfly_ugal has ugal routing, $x=4, y=4$.

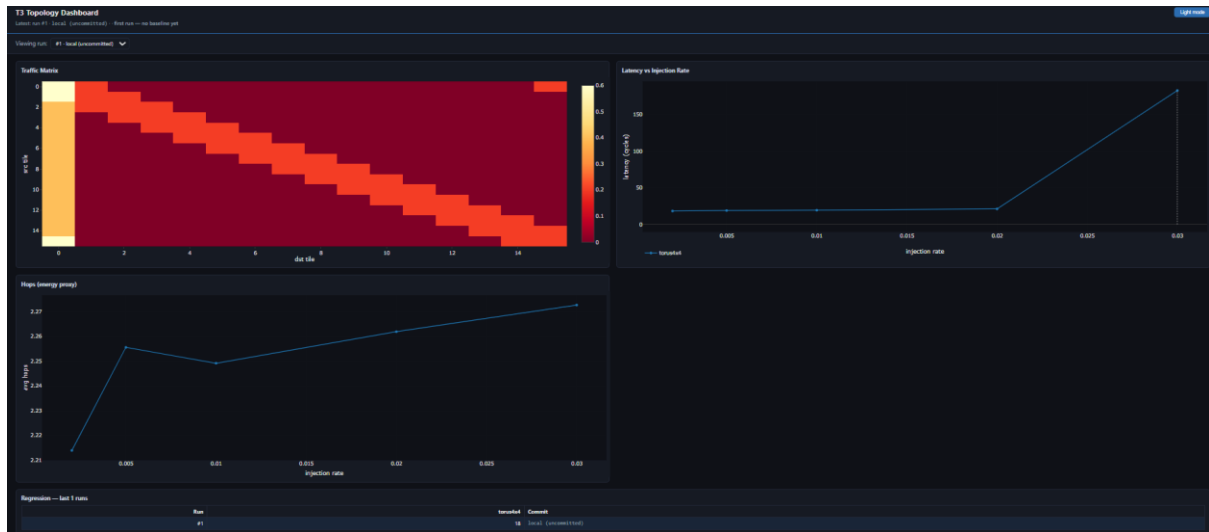
Task4: Add cmesh16.cfg (k=2, n=2, c=4, x=y=2, xr=yr=2)



topology	injection_rate	latency_cycles	hops_avg
cmesh16	0.002	14.8558	1.6093
cmesh16	0.005	15.141	1.63248
cmesh16	0.01	16.044	1.64169
cmesh16	0.02	17.6895	1.59156
cmesh16	0.03	200.105	1.64379

CMesh improves network efficiency by concentrating multiple processing nodes onto fewer routers, reducing hop count, router count, and overall network area. This results in lower latency and energy consumption under light traffic. However, concentration increases traffic aggregation at routers, causing higher contention and earlier saturation under heavy traffic. Therefore, CMesh provides better area and power efficiency at the cost of reduced congestion tolerance compared with less concentrated topologies.

Task5: Torus deadlock-safe routing check: use dim_order (torus-aware), NOT plain dor

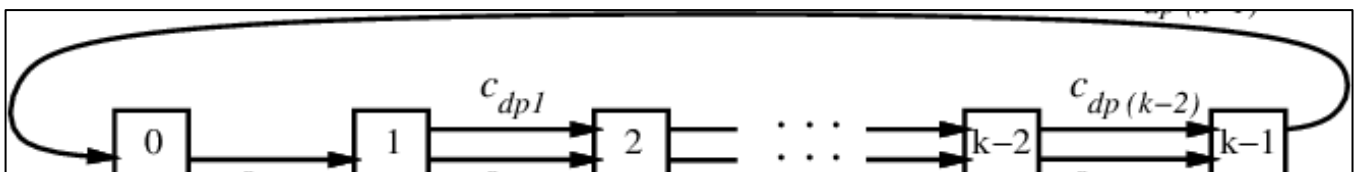


The Torus topology was configured with `routing_function = dim_order`, which is the torus-aware dimension-order routing algorithm. Unlike plain DOR, this implementation uses a dateline virtual-channel mechanism to break cyclic channel dependencies created by wrap-around links, ensuring deadlock-free packet routing.

The topology was evaluated across the specified injection-rate sweep (0.002–0.03). The simulation completed successfully for all tested loads without hanging or reporting routing errors, indicating that the routing algorithm remained deadlock-free under the evaluated traffic conditions.

`dim_order` executes successfully, whereas `dor` does not. This confirms that the Torus implementation in this Booksim build requires the torus-aware dimension-order routing function, which incorporates deadlock avoidance using the dateline virtual-channel mechanism. Plain DOR is unsupported for Torus in this implementation.

A Torus forms a cyclic channel dependency because of its wraparound links. To eliminate deadlock, the routing algorithm introduces a **logical dateline**, which is a designated wraparound boundary rather than a physical cut. Whenever a packet crosses this boundary, it transitions to a higher-priority Virtual Channel (e.g., VC0 → VC1). Since packets are never allowed to transition back (VC1 → VC0), the channel dependency graph becomes acyclic, guaranteeing deadlock-free routing while preserving minimal paths.



Deadlock in Torus

A **deadlock** occurs when packets form a **circular dependency**, where each packet holds a channel or virtual channel (VC) while waiting for another that is occupied by a different packet. Since no packet can progress, the network reaches a permanent stall.

Dateline

A **dateline** is a **logical boundary** placed on one wraparound link of each torus dimension. It is not a physical cut in the network, but a routing rule used to eliminate cyclic channel dependencies.

Virtual Channels (VCs)

A **Virtual Channel (VC)** is an independent logical buffer that shares the same physical link. When a packet crosses the dateline, it is required to transition from **VC0 to VC1**. Since packets are not allowed to transition back from **VC1 to VC0**, the dependency graph becomes acyclic, guaranteeing **deadlock-free routing**.

Torus Dimension-Order Routing

In a Torus, **Dimension-Order Routing (DOR)** computes the **minimum cyclic distance** in each dimension, allowing packets to use wraparound links when they provide a shorter path. Unlike Mesh routing, Torus DOR incorporates **dateline and Virtual Channel management** to safely handle cyclic links while preserving deadlock freedom.

Task6: Evaluate dragonflynew feasibility: n must be 1; no k gives 16 nodes ($k=2 \rightarrow 72$)

Item	Observation
Topology implementation	DragonFlyNew in networks/dragonfly.cpp
Supported routing	min, ugal
Constraint	$n \neq 1$ enforced by assertion
Network size	Computed as $(N = 2p^2(2p^2+1))$
Smallest supported network	72 nodes ($k = 2$)
16-node accelerator support	Not possible
Timeloop compatibility	Requires regenerating a 72×72 traffic matrix
UGAL requirement	Configure 3 virtual channels
Decision	No-Go for the current 16-node design; suitable for future larger-scale networks

Doesn't work for 16 nodes traffic matrix. Need to modify the traffic matrix for it to be viable for work, preferably a 72 nodes matrix.

Phase 1: Anynet

Task7: Learn anynet: arbitrary topology from adjacency file, routing min_anynet

Feature	Observation
Topology generation	User-defined
Input	Network/adjacency file
Parameters	Very few; topology comes from file
Routing	min_anynet
Best use	Rapid prototyping of custom NoCs
Advantage	No C++ coding required for topology changes
Limitation	Performance depends entirely on supplied graph

Successfully verified that Booksim can simulate arbitrary topologies using AnyNet. Unlike parameterized topologies (Mesh, Torus, Dragonfly), AnyNet constructs the network from an external graph description. This provides a convenient bridge for evaluating synthesized or custom NoC designs before implementing them as native C++ topology classes.

Unlike parameterized topologies such as Mesh or Dragonfly, AnyNet constructs the network entirely from a topology file. During initialization, it computes the router and node counts, instantiates routers with variable radix, connects all links, and automatically builds shortest-path routing tables using Dijkstra's algorithm. During simulation, packets are forwarded using the precomputed routing table via the `min_anynet` routing function. This makes AnyNet an ideal framework for evaluating custom-generated topologies before implementing them as native C++ network classes.

Task8: Adjacency-list → anynet network_file converter in Python

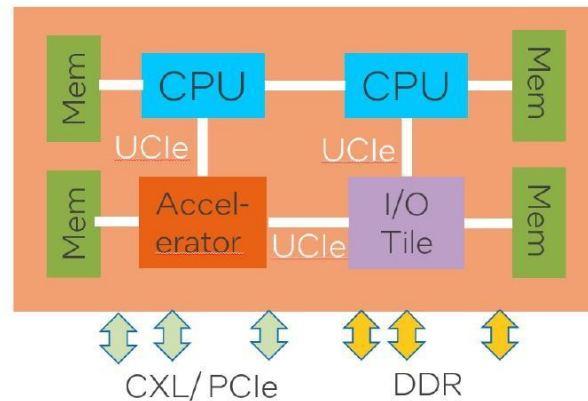
Added Info:

MEC Architecture:

UCIe:

CPU
↓
Generate packet
↓
Inject into Mesh
↓
Mesh Routing
↓
Reach Edge Router
↓
UCIe Adapter
↓
Serialize
↓
PHY
↓
Package Link
↓
PHY
↓
Deserialize
↓
Adapter
↓
Destination Mesh
↓
Destination Core

Figure 1. A Package comprising of CPU dies, Accelerator die(s), and I/O Tile die connected through UCIe



知乎 @乱up24

[Universal Chiplet Interconnect Express \(UCIe\): An Open Industry Standard for Innovations With Chiplets at Package Level | IEEE Journals & Magazine | IEEE Xplore](#)

Parameter	Effect
Number of lanes	Higher bandwidth
Link width	More bits/cycle
PHY frequency	Faster transfer
Adapter buffer size	Better throughput
Retry buffer	Reliability
Credit count	Flow control
Protocol	PCIe/CXL/Raw
Serialization width	Latency
Packaging technology	Delay & energy
Number of chiplets	Scalability

